Kartylicious: quality manual

Margarida Pinheiro [125011], Rodrigo Sampaio [125274], Miguel Sousa [125624]
v2026-11-03

1 Practices compliance 2
2 Project management 3
2.1 Assigned roles 3
2.2 Backlog grooming and progress monitoring 3
2.3 Requirements coverage 3

3 Code quality management 3
3.1 Guidelines for contributors 3
3.2 Code review 3
3.3 Static analysis and quality gates 4
3.4 3.4 Security 4

4 4 Continuous delivery pipeline (CI/CD) 4
4.1 4.1 Development workflow 4
4.2 CI/CD pipeline and tools 4
4.3 System observability 4
4.4 Artifacts repository (optional) 4

5 Continuous testing 5
5.1 Overall testing strategy and test development practice 5
5.2 Developer-facing tests (unit, integration, API) 5
5.3 5.4 End-to-end tests 5
5.4 Acceptance testing and BDD 5
5.5 Non-functional testing 5
5.6 Other test techniques 5
5.7 Exploratory testing 5

6 AI and agents in quality engineering 6
6.1 Team policy for the use of AI 6
6.2 Declaration of AI use 6
6.3 Agent instructions and coordination 6
6.4 Specification-driven generation 6
6.5 Agents in the CI pipeline 6
6.6 Critical analysis of AI's contribution 6

[This manual tells a new team member which QA practices the team follows and why, and proves that each one is in place.
• Guidelines: state the team's practices clearly and concisely, so that others can follow them.
• Rationale and evidence: explain why each practice was chosen and link to the evidence in the text (pipeline runs, dashboards, Jira and Xray reports, pull requests), so the reader can open it in context.
• Compliance: section 1 lists every practice of the guidelines (section 5) with its evidence
• The manual grows with the project.
• Use English or Portuguese; do not mix languages.
• Remove these tips, and this section, from your manual.]

1 Practices compliance analysis

To complete along the project and review by the end of the project.
Tip: One row per practice of the guidelines: E = expected, V = valued. Set the status (Done, Partial, Not adopted) and link to the evidence. A practice without an evidence link does not count. For valued practices you did not adopt, leave the status as Not adopted.

ID	Practice	Level	Status	Evidence/Notes
PM1	Scrum-like management in Jira	E	Done	https://kartylicious.atlassian.net/jira/software/projects/SCRUM/boards/1
PM2	User stories with points and acceptance criteria	E		
PM3	Jira linked to Git	E	Done	https://kartylicious.atlassian.net/jira/software/projects/SCRUM/development
PM4	Burndown and velocity	E		
PM5	Requirements coverage in Xray	E		
T1	BDD: Gherkin scenarios automated with Cucumber	E		
T2	Unit tests for business logic	E		
T3	Integration tests, run separately	E		
T4	End-to-end tests of the web UI	E		
T5	SLOs checked by a k6 load test	E		
T6	Test-driven development	V		
T7	Pairwise testing	V		
T8	Automated accessibility checks	V		
T9	GBFS feed contract test	V		
T10	Further non-functional tests	V		
Q1	Static analysis and coverage dashboard	E		
Q2	Blocking quality gate on new code	E		
Q3	Dependency vulnerability scanning	E	Done	./github/dependabot.yml file
Q4	Peer code review	E		
Q5	Contributor guidelines	E		

ID	Practice	Level	Status	Evidence/Notes
Q6	Container image scanning or added security testing	V		
D1	Feature branches, protected main, merge by pull request	E	Done	Ruleset created, main and develop branches protected. PR needed to branch with 1 reviewer
D2	CI with quality gate, badge and failure notification	E	*In progress*	*CI for basic project skeleton (spring-boot starter) setup*
D3	Infrastructure as code	E		
D4	Continuous delivery with representative data	E		
D5	Separate staging and production	V		
D6	Observability with SLO alerts	V		
AI1	Team AI policy and declaration of AI use	E		
AI2	Versioned agent instructions	E		
AI3	AI-assisted code review	E		
AI4	Specification-driven generation (≥ 2 stories)	E		
AI5	Critical analysis of AI's contribution	E		
AI6	Coordination of several agents	V		
AI7	Agents in the CI pipeline	V		

2 Project management

2.1 Assigned roles

Role	Team member
Team Coordinator	Miguel Sousa
Product Owner	Margarida Pinheiro
QA Engineer	Margarida Pinheiro
DevOps Master	Rodrigo Sampaio

2.2 Backlog grooming and progress monitoring

Tip: Do not explain the concepts; describe how the team works in routine with agile project management based on user stories. When is the backlog groomed? How are stories estimated and accepted? How is Jira linked to Git? How is progress tracked (burndown, velocity)? Link the board and the reports.

2.3 Requirements coverage tracking

Tip: How are tests linked to stories in Xray, and how do CI test results reach Xray? Link the coverage report and comment on any story without tests.

3 Code quality management

3.1 Guidelines for contributors

Tip: Define the coding style. Do not be exhaustive: highlight a few key choices and point to a fuller reference the team adopted (for example, the AOSP Java code style). These guidelines must match the agent instructions in the repository.

3.2 Code review practice

Tip: When is a review required, and who approves? Give a short review checklist. Explain how AI-assisted review is used on pull requests (tool, what it checks), how the human reviewer decides on its findings, and where accepted and rejected findings are recorded.

3.3 Static analysis and quality gates

Tip: Describe the static analysis setup (for example, SonarQube with JaCoCo coverage) and link the dashboard. Which quality gate conditions were defined, on new code and overall? What is the rationale for each threshold? Show evidence that a failing gate blocks the merge.

3.4 Other rules in code inspection

Security
Tip: How are vulnerabilities found in code (security issues and hotspots in static analysis) and in dependencies (for example, Dependabot or OWASP Dependency-Check)? Who reviews the findings, and when must they be fixed? Mention container image scanning or other security tests if adopted.

Accessibility
Tip: discuss, if applicable.

Compliance with xxx
Tip: discuss, if applicable.

4 Continuous delivery pipeline (CI/CD) and observability

4.1 Development workflow

Coding workflow
Tip: Explain to a newcomer how a developer takes a story and carries it to main. Which workflow does the team follow (for example, GitHub flow or Gitflow), and how do branches map to stories? Which branch protection rules are set? How do you integrate AI?

Definition of done
Tip: What is the team's definition of done for a user story? State which tests, reviews and gates are required.

4.2 CI/CD pipeline and tools

Tip: Describe the continuous integration pipeline: triggers, stages, tools and their configuration, how failures reach the author, the build badge. Then describe continuous delivery: the containers, the infrastructure-as-code files, the environments (staging, production) and how main is deployed with representative data. Link the pipeline definitions and recent runs.

4.3 System observability

Tip: What was set up for proactive monitoring of the system's operating conditions? Which data is collected, which dashboards exist, which alerts fire, and how do they relate to the SLOs of the Product Specification report?

4.4 Artifacts repository (optional)

Tip: Describe how build artifacts (Maven packages, container images) are stored and versioned, if the team uses a repository for them.

5 Continuous testing

5.1 Overall testing strategy and test development practice

Tip: What is the overall test strategy, and how is testing aligned with CI/CD? Which kinds of test run on every push, which on merge to main, which before a release? Did the team use TDD or BDD, and which tools were combined (for example, Cucumber with REST Assured)? Do not list the tests themselves: explain the policies and show evidence that test results gate the pipeline.

5.2 Developer-facing tests (unit, integration, API)

Tip: Policy for unit tests (open box): when must a developer write them, and which are the most relevant ones in the project? Policy for integration tests: persistence (for example, Testcontainers), the fleet simulator adapter and its contract tests, the payment sandbox, the REST API. How are integration tests kept apart from unit tests in the build? If TDD was used, show the evidence.

5.3 Acceptance testing

Tip: Policy for acceptance tests (closed box, user-facing): which stories need Gherkin scenarios, who writes them and when, and how they are automated.

Clarify whether you are using web UI automation (Selenium or Playwright), with or without Cucumber.
How are tests linked to Xray?

5.4 Non-functional testing

Tip: Policy for performance tests: which SLOs are checked, with which k6 scenarios and thresholds, and where the results are. Add stress or soak tests and accessibility checks if adopted.

5.5 Other test techniques

Tip: Pairwise or combinatorial testing (which variables, which tool, which risk it addresses); contract test of the GBFS feed against the official schema; other techniques the team found useful.

5.6 Exploratory testing

Tip: Strategy for unscripted tests, if any: sessions, charters, and how findings became automated tests or backlog items.

6 AI and agents in quality engineering

6.1 Team policy for the use of AI

Tip: State the team's position on AI assistants and agents for production code, tests, documentation and reviews, within the course rules (guidelines, section 6.2). Be clear about the do's and don'ts, and give practical advice to a newcomer.

6.2 Declaration of AI use

Tip: One declaration for the whole project. List each tool or agent, what it was used for and how its output was verified.

Tool or agent (and model)	Used for	How the output was verified

6.3 Agent instructions and coordination

Tip: Link the agent instructions in the repository (AGENTS.md or the tool's equivalent) and summarize what they cover: style, definition of done, test rules, build commands. Who maintains them, and how do they stay consistent with this manual? If several agents work together, describe their roles, the handoffs between them and the human checkpoints.

6.4 Specification-driven generation

Tip: For each story generated from its specification, link the specification (Gherkin scenarios and/or OpenSpec change proposal), the agent's pull request and the human review. What had to be corrected, and why? Link to pull requests for evidence.

6.5 Agents in the CI pipeline

Tip: If adopted: which tasks do agents perform in the pipeline (summarizing pull requests, triaging failures, detecting flaky tests, suggesting fixes)? How are their outputs used, and what stops them from merging anything?

6.6 Critical analysis of AI's contribution

Tip: Assess with data where AI helped and where it failed. Some topics: share of merged changes produced mostly by AI; AI review findings accepted / rejected; defects or rework traced to AI output; Quality gate failures on AI-produced changes;…
Close with what the team would do differently.